

AI ENGINEERING — WEEK 1

Python Foundations: The Complete Reading Guide

Environment setup, core syntax, modules, control flow, data structures, functions, and comprehensions — a complete foundation to read at your own pace before Week 2.

Companion reading to the Week 1 lecture slides. Lab exercises are not included here — see the separate lab handout.

Chapter 1 — Setting Up Your Python Environment

By the end of this chapter you should be able to

- Install Python on your own machine and confirm the install worked
- Set up a code editor that's actually using the right interpreter
- Create, activate, and exit an isolated virtual environment

1.1 Installing Python

Before you can run any of the code in this guide, Python needs to be installed on your machine. There are a couple of reliable ways to do this, depending on your operating system.

Windows

Download the installer from python.org/downloads. During installation, check the box that says **"Add Python to PATH"**. This single checkbox is the most common source of installation problems — if it's unchecked, your terminal won't know where to find Python, and you'll get a "command not found" error even though Python is technically installed.

macOS

You have two solid options: the installer from python.org, or Homebrew if you already have it (`brew install python@3.12`). Either works fine — pick whichever is easier for you right now.

Verifying the install

Once installed, open a terminal and run:

```
python3 --version
```

You should see something like `Python 3.12.11`. If instead you get "command not found," the most likely causes are: the PATH checkbox wasn't checked (Windows), or you need to close and reopen your terminal so it picks up the new installation.

1.2 Virtual Environments

A virtual environment gives each project its own isolated Python interpreter and its own set of installed packages, so dependencies from one project never collide with another. You'll use this workflow constantly, so it's worth understanding thoroughly rather than just memorizing the commands.

```
# check python version
python3 --version

# create an isolated environment named .venv
python3.12 -m venv .venv

# activate it (macOS/Linux)
source .venv/bin/activate

# activate it (Windows PowerShell)
.venv\Scripts\Activate.ps1

# install a package into the active venv
pip install requests

# confirm which interpreter is currently in use
which python
```

When a virtual environment is active, your terminal prompt will usually show its name in parentheses, like `(.venv)` `yourname@laptop`. That's your visual cue that anything you install right now goes into this project's isolated environment, not your system-wide Python.

Leaving a virtual environment

Just as important as activating a venv is knowing how to exit one. When you're done working in a project, run:

```
deactivate
```

This returns you to your system's default Python. Forgetting this step usually isn't harmful, but it can lead to confusion later if you

open a new project and wonder why packages "aren't installed" — they were installed into the previous project's venv, not this one.

1.3 Choosing an Editor

The recommended default for this course is **VS Code**, paired with its official Python extension. A few other options exist — PyCharm and Jupyter notebooks both have their place — but VS Code is a solid, free, widely-used starting point.

The #1 setup mistake: writing code in an editor that isn't actually pointed at your virtual environment's interpreter. In VS Code, use the "Select Interpreter" command (accessible from the command palette or the bottom status bar) and make sure it's pointing at the Python inside your `.venv` folder, not your system Python. If package imports mysteriously fail even though you installed them, this is almost always the cause.

1.4 Common Setup Pitfalls

- **"pip not found":** usually means your venv isn't activated, or Python wasn't added to PATH.
- **python vs python3 :** on some systems, `python` points to an old Python 2 install or nothing at all. When in doubt, use `python3` explicitly.
- **Installing packages before activating the venv:** if you run `pip install` without activating your venv first, the package goes into your system Python instead of your project's isolated environment.

Chapter 2 — Syntax Basics

By the end of this chapter you should be able to

- Name variables and functions following Python convention
- Use arithmetic operators and convert between types deliberately
- Work with strings directly — indexing, slicing, common methods, and number formatting
- Accept input from a user and understand `None` as a distinct value
- Read a traceback well enough to find the line that actually failed

2.1 Naming Things & Reserved Words

Variable and function names in Python follow `snake_case` convention — lowercase words separated by underscores, like `student_name` or `calculate_average`. Names must start with a letter or underscore, and can't contain spaces or most punctuation.

Python also reserves certain words that can't be used as variable names because they already mean something to the language: `class`, `for`, `if`, `while`, `return`, `def`, `import`, `True`, `False`, `None`, and a handful of others. If you try to write `class = "CS101"`, Python will raise a `SyntaxError` because `class` is reserved for defining classes.

2.2 Variables & Dynamic Typing

Python is **dynamically typed**. A name is bound to a value with `=`, and there's no type declaration required — the type lives with the value, not the variable name itself.

```
enrolled = 42
gpa = 3.7
course = "Intro to Python"
is_open = True

print(type(enrolled))    # <class 'int'>
print(enrolled)          # 42

enrolled = "forty-two"
print(type(enrolled))    # <class 'str'>
print(enrolled)          # forty-two
```

Notice that the same variable, `enrolled`, held an `int` and then a `str` — nothing about the variable itself was ever declared as one type. The four core built-in types you'll use constantly are `int`, `float`, `str`, and `bool`.

2.3 Arithmetic Operators & Type Casting

Python's arithmetic operators are mostly what you'd expect, with two that trip people up the first time they see them:

Operator	Meaning	Example
<code>+</code>	addition	<code>3 + 2 → 5</code>
<code>-</code>	subtraction	<code>3 - 2 → 1</code>
<code>*</code>	multiplication	<code>3 * 2 → 6</code>
<code>/</code>	true division (always returns a float)	<code>7 / 2 → 3.5</code>
<code>//</code>	floor division (truncates to an integer)	<code>7 // 2 → 3</code>
<code>%</code>	modulo (remainder)	<code>7 % 2 → 1</code>
<code>**</code>	exponent	<code>2 ** 3 → 8</code>

Common surprise: `7 // 2` returns `3`, not `3.5`. Floor division always truncates toward the nearest whole number below the true result, even with negative numbers — `-7 // 2` is `-4`, not `-3`.

Because Python is dynamically typed, it's easy to accidentally mix types in a way that raises an error:

```
age = 20
message = "I am " + age + " years old"
# TypeError: can only concatenate str (not "int") to str
```

The fix is **type casting** — explicitly converting a value from one type to another using `int()`, `str()`, `float()`, or `bool()`:

```
age = 20
message = "I am " + str(age) + " years old"
print(message)  # I am 20 years old
```

2.4 Strings, Properly

Strings are one of Python's core types, and they support the same slicing syntax you'll see later with lists.

```
name = "Amara"
print(name[0])    # 'A' – first character
print(name[-1])   # 'a' – last character
print(name[1:4])  # 'mar' – characters from index 1 up to (not including) 4
```

Strings are immutable. Unlike lists, you can't change a string in place — `name[0] = "X"` raises a `TypeError`. Every string method (like `.upper()` or `.replace()`) returns a brand-new string rather than modifying the original. If you want the change to stick, you have to reassign it: `name = name.upper()`.

Common string methods

```
text = "  Hello World  "
text.strip()          # "Hello World" – removes leading/trailing whitespace
text.strip().split(" ") # ["Hello", "World"] – splits into a list on a separator
"-".join(["a", "b", "c"]) # "a-b-c" – joins a list into a single string
text.upper()          # "  HELLO WORLD  "
text.lower()          # "  hello world  "
text.replace("World", "Python") # "  Hello Python  "
```

f-strings

You've likely already seen f-strings used in example code without a formal introduction — they're the modern, readable way to build strings that include variable values:

```
name = "Amara"
score = 92
print(f"{name} scored {score}")  # Amara scored 92
```

Older styles like `.format()` and `%`-formatting still exist and you may see them in other people's code, but f-strings are the default going forward.

2.5 `input()` and `None`

`input()` pauses your program and waits for the user to type something, then returns whatever they typed — **always as a string**, even if they type a number.

```
age_text = input("How old are you? ")
age = int(age_text)  # cast to int before doing math with it
print(f"Next year you'll be {age + 1}")
```

`None` is Python's way of representing "no value here." It's not the same as `0`, `""`, or `False` — those are all real values that happen to be falsy (more on that in Chapter 4). `None` specifically means "nothing was assigned."

```
result = None
print(result)      # None
print(type(result)) # <class 'NoneType'>
```

2.6 Reading a Traceback

You'll hit errors constantly while learning — that's normal. What matters is being able to read the error message (called a **traceback**) well enough to find the actual problem.

```
Traceback (most recent call last):
  File "main.py", line 7, in <module>
    print(total / count)
ZeroDivisionError: division by zero
```

Read from the bottom up. The last line tells you the *type* of error (here, `ZeroDivisionError`) and usually a short description. The line just above it points to the exact line in your code where it happened. It's tempting to start reading from the top, but the most useful information is at the bottom.

2.7 Multiple Assignment & Unpacking

Python lets you assign several variables in a single line:

```
a, b = 1, 2
print(a, b)    # 1 2
```

This enables a classic swap idiom that doesn't need a temporary variable, unlike most other languages:

```
a, b = 5, 10
a, b = b, a
print(a, b)    # 10 5
```

2.8 Comments

Anything after a `#` on a line is ignored by Python — comments are for humans reading the code, not for the interpreter. Use them to explain *why* something is done, not to restate what the code already says plainly.

```
# bad — just restates the code
x = x + 1 # add one to x

# good — explains a non-obvious reason
x = x + 1 # offset for the header row we skipped earlier
```

2.9 Escape Sequences & Multi-line Strings

Certain characters can't be typed directly into a string and need an escape sequence instead:

Sequence	Meaning
<code>\n</code>	newline
<code>\t</code>	tab
<code>\\</code>	a literal backslash
<code>\"</code>	a literal double quote inside a double-quoted string

```
print("Line one\nLine two")
# Line one
# Line two

path = "C:\\Users\\Amara"    # each \\ becomes a single literal backslash
```

For strings that span multiple lines naturally, use triple quotes instead of stacking `\n` :

```
message = """Dear student,

Welcome to the course.
"""
```

If you want a string where backslashes are treated literally — common in file paths or regular expressions — prefix it with `r` for a "raw" string:

```
path = r"C:\Users\Amara"    # no need to double the backslashes
```

2.10 `is` vs `==`

`==` checks whether two values are **equal**. `is` checks whether two names point to the **exact same object** in memory. For everyday value comparisons, you almost always want `==` .

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)    # True – same contents
print(a is b)    # False – two different list objects that happen to look alike
```

The one place `is` is correct: comparing against `None`. Always write `if x is None:` rather than `if x == None:` — it's the accepted convention, and it avoids edge cases with custom objects that redefine what `==` means.

2.11 Operator Precedence & Short-Circuit Evaluation

Just like in math, some operators bind tighter than others. From highest to lowest priority (roughly): `**`, then `* / // %`, then `+`, `-`, then comparisons (`==` `!=` `<` `>` etc.), then `not`, then `and`, then `or`.

```
print(2 + 3 * 4)    # 14, not 20 – multiplication happens first
print((2 + 3) * 4)  # 20 – parentheses override precedence
```

When in doubt, add parentheses. They cost nothing and remove any ambiguity for whoever reads the code next — including you, in six months.

Python also **short-circuits** `and` / `or`: it stops evaluating as soon as the overall result is already determined.

```
def check():
    print("checked!")
    return True

# check() never runs – the left side of 'and' is already False,
# so Python already knows the whole expression is False
result = False and check()
```

2.12 Common Exception Types — A Field Guide

You'll meet these constantly. Recognizing the type of error at a glance saves a lot of guesswork. Full exception *handling* (catching these with `try / except`) comes in Week 2 — for now, this is about recognizing what each one means.

Exception	What usually causes it
<code>TypeError</code>	Using a value of the wrong type, e.g. <code>"5" + 5</code>
<code>ValueError</code>	Right type, invalid value, e.g. <code>int("abc")</code>
<code>NameError</code>	Using a variable that was never defined, or a typo in its name
<code>IndexError</code>	Accessing a list index that doesn't exist, e.g. <code>my_list[10]</code> on a 3-item list
<code>KeyError</code>	Accessing a dictionary key that doesn't exist
<code>AttributeError</code>	Calling a method that doesn't exist on that type, e.g. <code>"hello".push()</code>
<code>ZeroDivisionError</code>	Dividing by zero
<code>ImportError</code> / <code>ModuleNotFoundError</code>	Importing a module that isn't installed or doesn't exist
<code>IndentationError</code>	Inconsistent or incorrect indentation

2.13 Formatting Numbers & Floating-Point Precision

f-strings support format specs for controlling how a number displays — a colon followed by the format, right inside the braces:

```
price = 19.999
print(f"{price:.2f}")    # 20.00 – round to 2 decimal places
print(f"{1234567:,}")    # 1,234,567 – thousands separator
```

`round()` works the same way outside of an f-string, when you need the rounded number itself rather than just a display string:

```
print(round(19.999, 2))  # 20.0
```

The classic float surprise:

```
print(0.1 + 0.2)          # 0.30000000000000004, not 0.3
print(0.1 + 0.2 == 0.3)   # False
```

This isn't a Python bug — it's how floating-point numbers work in nearly every programming language, because some decimal fractions can't be represented exactly in binary. Never compare floats with `==` directly. Instead, check that they're close enough:

```
print(round(0.1 + 0.2, 10) == round(0.3, 10))  # True
```

Chapter 3 — Modules & Imports

By the end of this chapter you should be able to

- Explain what a module is and why code is split across them
- Import a whole module, specific names from a module, or a module under an alias
- Name a few standard library modules you'll actually use

3.1 What a Module Is

A module is simply a Python file containing code — functions, classes, variables — that you can reuse in another file instead of retyping it. Python ships with a large standard library of built-in modules, and you can also install third-party ones (like `requests`, which you installed with `pip` back in Chapter 1) or write your own.

3.2 The `import` Statement

```
# import an entire module, access things with module.name
import math
print(math.sqrt(16))    # 4.0
print(math.pi)          # 3.141592653589793

# import only specific names directly into your namespace
from math import sqrt, pi
print(sqrt(16))          # 4.0 — no need to prefix with math.

# import a module under a shorter alias — very common with third-party libraries
import numpy as np
```

Which style to use? `import module` keeps things explicit about where a function came from (`math.sqrt`), which is usually clearer in larger files. `from module import name` is fine for a small number of frequently used names. Avoid `from module import *` — it pulls in everything and makes it unclear where any given name actually came from.

3.3 A Few Standard Library Modules Worth Knowing

Module	What it's for
<code>math</code>	Mathematical functions and constants (<code>sqrt</code> , <code>pi</code> , <code>floor</code> , <code>ceil</code>)
<code>random</code>	Generating random numbers and choices (<code>random.randint</code> , <code>random.choice</code>)
<code>datetime</code>	Working with dates and times
<code>os</code>	Interacting with the operating system — file paths, environment variables
<code>json</code>	Reading and writing JSON data

```
import random
print(random.randint(1, 6))    # a random integer from 1 to 6, inclusive
```

Chapter 4 — Truthiness, Operators & Control Flow

By the end of this chapter you should be able to

- Use comparison and logical operators correctly
- Predict how Python evaluates non-boolean values in an `if` statement
- Write conditional logic and loops confidently, including ternary expressions
- Use `enumerate()` and `zip()` , and use `range()` with a step

4.1 Comparison & Logical Operators

Operator	Meaning
<code>==</code>	equal to
<code>!=</code>	not equal to
<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	less/greater than (or equal to)
<code>and</code>	true only if both sides are true
<code>or</code>	true if either side is true
<code>not</code>	flips a boolean value

Python also allows chained comparisons, which read naturally and are unique to Python compared to many other languages:

```
age = 25
print(18 <= age < 65)    # True – equivalent to (18 <= age) and (age < 65)
```

4.2 Truthiness — What "Branching on Truth" Actually Means

An `if` statement doesn't strictly require a `True` / `False` value — Python will evaluate almost anything for "truthiness." A specific set of values are considered **falsy**:

```
0, 0.0, "", None, [], {}, set()
```

Everything else counts as **truthy** — including every non-empty string, non-zero number, and non-empty collection.

```
my_list = []

if my_list:
    print("has items")
else:
    print("empty")    # this prints – an empty list is falsy

if my_list is not None:
    print("list exists")    # this ALSO prints – the list isn't None, it's just empty
```

Why this matters: `if my_list:` and `if my_list is not None:` are checking two different things. The first asks "does this have anything in it?" The second asks "was this ever set to None?" An empty list passes the second check but fails the first — mixing these up is a common source of subtle bugs.

4.3 Conditionals

Python uses indentation — not braces — to define code blocks. Each `elif` is only checked if every condition above it was false.

```
def letter_grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    else:
        return "F"

print(letter_grade(87))    # B
```

4.4 Conditional (Ternary) Expressions

For simple cases, an `if / else` can be written as a single-line expression — you'll see this constantly in other people's code:

```
age = 20
status = "adult" if age >= 18 else "minor"
print(status)    # adult
```

This is exactly equivalent to a full if/else block that assigns one of two values — it's just more compact when both branches are short:

```
# the long way
if age >= 18:
    status = "adult"
else:
    status = "minor"

# the ternary way – same result
status = "adult" if age >= 18 else "minor"
```

This only replaces simple value-assignment if/else pairs, not full blocks with multiple statements. If either branch needs more than one line, use a regular if/else instead.

4.5 Loops

`for` iterates over a known sequence. `while` repeats until a condition becomes false. `break` exits a loop early; `continue` skips straight to the next iteration.

```
for week in range(1, 3):
    print(f"Week {week} starts")

attempts = 0
while attempts < 3:
    attempts += 1
    if attempts == 2:
        continue
    print(f"Attempt {attempts}")
```

4.6 `enumerate()` and `zip()`

Two small tools that make loops significantly more useful once you're working with real data.

`enumerate()` — index and value together

```
students = ["Amara", "Leo", "Priya"]

# manual counter (works, but clunky)
i = 0
for name in students:
    print(i, name)
    i += 1

# enumerate() does this for you
for i, name in enumerate(students):
    print(i, name)

# start counting from 1 instead of 0
for i, name in enumerate(students, start=1):
    print(i, name)
```

zip() — looping over two sequences together

```
students = ["Amara", "Leo", "Priya"]
scores = [92, 85, 78]

for name, score in zip(students, scores):
    print(f"{name}: {score}")
```

Watch out: `zip()` stops at the shortest sequence. If `students` has 3 names but `scores` only has 2 numbers, the third student is silently dropped — no error is raised.

4.7 `range()` with a Step, and `sorted()` vs `.sort()`

`range()` accepts a third argument for stepping by more than one, including negative steps to count backward:

```
for n in range(0, 10, 2):
    print(n)    # 0, 2, 4, 6, 8

for n in range(10, 0, -1):
    print(n)    # counts down: 10, 9, 8, ... 1
```

You've seen `.sort()`, which sorts a list in place and returns `None`. `sorted()` is the built-in function version — it works on any iterable and returns a **new** sorted list, leaving the original untouched:

```
scores = [92, 78, 85]
ranked = sorted(scores, reverse=True)
print(ranked)    # [92, 85, 78]
print(scores)    # [92, 78, 85] — unchanged
```

4.8 Debugging Control Flow: From Research to Practice

A 2023 study cataloguing novice programming misconceptions (MDPI Computers, "Explain Trace: Misconceptions of Control-Flow Statements") found that control-flow statements cause broad, frequently-overlooked confusion among beginners — enough that researchers launched a dedicated effort to define these misconceptions precisely. Documented errors that show up repeatedly include assigning an expression instead of its computed value, and using a variable's previous value instead of its current one.

What matters practically is having a habit for catching these in your own code:

- When a loop or conditional isn't behaving as expected, add a temporary `print()` statement right before the suspicious line to confirm the actual current value of your variables — don't assume you know what it is.
- If a value seems "stuck" at an old value inside a loop, check whether you're updating it before or after you use it.
- Read error tracebacks from the bottom up (see 2.6), and don't be afraid to add throwaway `print()` statements liberally while debugging — you'll remove them once the code works.

Chapter 5 — Data Structures

By the end of this chapter you should be able to

- Choose the right structure (list, dict, set, or tuple) for a task
- Index, slice, and mutate lists and dictionaries confidently
- Safely access dictionary values without crashing on a missing key
- Use set operations to compare collections
- Read and modify nested structures — lists of dicts, dicts of lists

5.1 Lists & Tuples

Both are ordered sequences. Lists are **mutable**, meaning you can change them after creation. Tuples are **immutable** — fixed once built. Use tuples for fixed records (like coordinates); use lists for collections that grow or shrink.

```
# lists
students = ["Amara", "Leo", "Priya"]
students.append("Sam")
print(students[1:])    # ['Leo', 'Priya', 'Sam']

# tuple
point = (40.7, -74.0)
```

5.2 List Methods

```
fruits.append("mango")    # add to the end
fruits.insert(1, "kiwi")  # insert at a specific index
fruits.pop()              # remove and return the last item
fruits.pop(0)             # remove and return item at index 0
fruits.remove("date")     # remove first matching value (not by index)
fruits.index("cherry")    # find the index of a value
fruits.sort()             # sort in place, ascending
fruits.reverse()          # reverse in place
len(fruits)               # number of items
"apple" in fruits         # membership check (True/False)
```

There's also the `del` statement, which removes an item by index without returning it (unlike `.pop()`):

```
del fruits[0]    # just removes it – nothing is returned
```

5.3 Dictionaries

Dictionaries map unique keys to values.

```
grades = {"Amara": 92, "Leo": 85}
grades["Priya"] = 78
print(grades["Leo"])    # 85
```

5.4 Dictionary Methods Beyond Bracket Access

Accessing a dictionary with brackets crashes if the key doesn't exist:

```
print(grades["Miguel"])
# KeyError: 'Miguel'
```

`.get()` is the safe alternative — it returns a default value (or `None`) instead of crashing:

```
print(grades.get("Miguel"))    # None
print(grades.get("Miguel", "N/A"))    # N/A
```

To loop over a dictionary's contents, use `.keys()`, `.values()`, or `.items()`:

```
for name in grades.keys():
    print(name)

for score in grades.values():
    print(score)

for name, score in grades.items():
    print(f"{name}: {score}")
```

`del` also works on dictionaries, to remove a key entirely:

```
del grades["Leo"]  # removes the key (and its value) completely
```

5.5 Sets & Set Operations

Sets store unique items with no order or duplicates. Both dictionaries and sets use hashing internally, so membership checks (`in`) are fast.

```
enrolled_ids = {101, 102, 103}
print(102 in enrolled_ids)  # True
```

Sets support union, intersection, and difference — useful whenever you need to compare two collections:

```
enrolled = {"Amara", "Leo", "Priya"}
waitlisted = {"Leo", "Sam"}

print(enrolled | waitlisted)  # union: everyone in either group
print(enrolled & waitlisted)  # intersection: {'Leo'} – in both
print(enrolled - waitlisted)  # difference: enrolled but not waitlisted
```

5.6 Deeper Slicing, Lists of Lists, and Sorting with a Key

Slicing supports negative indices and a step, on both lists and strings:

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(nums[-3:])  # [7, 8, 9] – last three items
print(nums[::2])  # [0, 2, 4, 6, 8] – every second item
print(nums[::-1])  # reversed list, without calling .reverse()
```

A list can hold other lists — this is how you represent a grid or table (sometimes called a matrix):

```
grid = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
]

print(grid[1])  # [4, 5, 6] – the second row
print(grid[1][2])  # 6 – row 1, column 2
```

Both `.sort()` and `sorted()` accept a `key` parameter — a function that decides what to sort by. This is especially useful with a list of dicts:

```
students = [
    {"name": "Amara", "grade": 92},
    {"name": "Leo", "grade": 85},
]

ranked = sorted(students, key=lambda s: s["grade"], reverse=True)
print(ranked[0]["name"])  # 'Amara' – highest grade first
```

That `lambda` is a small preview — you'll cover lambda functions properly in a later module. For now, just recognize the pattern: `key=` takes a function that tells `sorted()` what value to compare on.

5.7 Nesting Data Structures

Real data is rarely a single flat list or dict — it's usually a combination.

List of dicts

```
students = [
    {"name": "Amara", "grade": 92},
    {"name": "Leo", "grade": 85},
]

print(students[0]["grade"])    # 92
students[0]["grade"] = 95      # update a value
students.append({"name": "Priya", "grade": 78})    # add a new record
```

This exact shape — a list of dictionaries — is what you'll be sorting and filtering with lambda functions later in the course. Getting comfortable with it now pays off directly.

Dict of lists

```
history = {
    "Amara": [92, 85, 78],
    "Leo": [88, 91],
}

history["Amara"].append(90)    # add a new score for Amara
print(history["Leo"][0])      # 88 – Leo's first recorded score
```

5.8 Common Pitfalls with Data Structures

- **Copy vs. reference:** `b = a` does not create a new list — `b` and `a` point to the same data, so changing one changes the other. Use `b = a.copy()` for a shallow copy, or `copy.deepcopy(a)` if the structure contains nested lists/dicts.
- **KeyError on nested access:** reaching into a nested structure with a missing key at any level will crash — check with `.get()` at each level if the data isn't guaranteed to exist.

There's a third classic pitfall involving mutable objects — one specific to function default arguments — covered in Chapter 6.4 once functions with default values have been introduced.

Chapter 6 — Functions

By the end of this chapter you should be able to

- Explain the difference between a method and a function
- Define a function with parameters and use `return` to send back a result, including multiple values at once
- Use default arguments, keyword arguments, and `*args` / `**kwargs`
- Write a docstring and reason about local vs. global scope

6.1 Methods vs. Functions

Methods perform actions on a specific object (like `fruits.append(...)`). Functions are reusable blocks of code, defined once and called by name whenever you need them. Parameters let a function accept different inputs each time it's called.

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Amara")  
# 'name' is the PARAMETER  
# "Amara" is the ARGUMENT
```

Planning ahead: if you want to sketch out a function's name and parameters before writing its body, Python needs at least one line inside — an empty function is a `SyntaxError`. The `pass` statement is a placeholder that does nothing, just to keep the function valid while you stub it out:

```
def calculate_gpa(courses):  
    pass # TODO: implement this later
```

You'll use this constantly once you start sketching out classes in Week 2.

6.2 `return`

`return` sends a result back to whoever called the function, rather than just printing it. This matters because a printed value can't be reused elsewhere in your program, but a returned value can be stored and passed around.

```
def greet(name):  
    return f"Hello, {name}!" # sends the value back  
  
message = greet("Amara")  
print(message)
```

6.3 Returning Multiple Values

A function can return more than one value at once — Python packs them into a tuple automatically, which you can unpack right at the call site using the same syntax from Chapter 2.7:

```
def min_max(nums):  
    return min(nums), max(nums) # returns a tuple: (lowest, highest)  
  
lo, hi = min_max([4, 8, 1, 9, 3])  
print(lo, hi) # 1 9
```

This is an extremely common pattern once you start writing functions that need to hand back more than one piece of information — it's often cleaner than bundling everything into a dict just to return it.

6.4 Default Arguments & Keyword Arguments

A parameter can have a default value, used whenever the caller doesn't supply one:

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Amara"))           # Hello, Amara!
print(greet("Leo", "Welcome"))  # Welcome, Leo!
print(greet(name="Priya", greeting="Hey")) # calling by keyword – order no longer matters
```

The mutable default argument trap: never use a list or dict as a default value directly.

```
# DON'T do this
def add_student(name, roster=[]):
    roster.append(name)
    return roster

# that same empty list is reused across every call, so it silently
# accumulates students from previous, unrelated calls

# DO this instead
def add_student(name, roster=None):
    if roster is None:
        roster = []
    roster.append(name)
    return roster
```

6.5 `*args` and `**kwargs`

Sometimes you don't know in advance how many arguments a function needs to accept. `*args` collects any number of extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dictionary.

```
def total(*args):
    return sum(args)

print(total(1, 2, 3))          # 6
print(total(1, 2, 3, 4, 5))    # 15 – any number of arguments works

def describe(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

describe(name="Amara", grade=92)
# name: Amara
# grade: 92
```

6.6 Docstrings

A docstring is a string literal placed right after a function definition, documenting what it does. It's convention for anything beyond a trivial one-liner, and tools like `help()` can display it automatically.

```
def letter_grade(score):
    """Return a letter grade (A-F) for a numeric score out of 100."""
    if score >= 90:
        return "A"
    return "F"

print(letter_grade.__doc__) # prints the docstring itself
```

6.7 Scope — Local vs. Global

A variable created inside a function only exists inside that function — this is its **local scope**. It doesn't exist outside, and it doesn't automatically affect a same-named variable outside the function.

```
def set_score():
    score = 100 # local to this function only
    print(score)

set_score()
print(score) # NameError: score isn't defined out here
```

A common scoping bug: assuming a variable assigned inside a function changes the outer variable of the same name. It doesn't, unless you explicitly use the `global` keyword (which is rarely the right tool — passing values in and returning them out is almost always cleaner).

```
total = 0

def add(n):
    total = total + n    # UnboundLocalError – Python sees the assignment
                        # and treats total as local to this function,
                        # so it can't read the outer 'total' first

# the fix: return the new value instead of trying to mutate the outer one
def add(n, total):
    return total + n

total = add(5, total)
```

Chapter 7 — Comprehensions

By the end of this chapter you should be able to

- Rewrite a simple loop-and-append pattern as a list comprehension
- Read and write basic dict comprehensions
- Recognize when a comprehension has become less readable than a plain loop

7.1 List Comprehensions

A list comprehension is a compact way of writing a loop that builds a new list. The general pattern is:

```
[expression for item in iterable]
```

```
# the manual way
squares = []
for n in range(5):
    squares.append(n ** 2)

# the comprehension way – same result
squares = [n ** 2 for n in range(5)]
```

You can add a condition to filter which items are included:

```
nums = [1, 2, 3, 4, 5, 6]
evens = [x for x in nums if x % 2 == 0]
print(evens)    # [2, 4, 6]
```

7.2 Dict Comprehensions

The same idea applies to building dictionaries:

```
{key: value for item in iterable}
```

```
students = ["Amara", "Leo", "Priya"]
scores = [92, 85, 78]

grade_map = {name: score for name, score in zip(students, scores)}
print(grade_map)    # {'Amara': 92, 'Leo': 85, 'Priya': 78}
```

7.3 When Not to Use a Comprehension

Comprehensions are meant to improve readability, not sacrifice it. If a comprehension needs a second condition or a nested loop to express what you want, it's usually clearer to write it as a regular loop instead.

```
# technically valid, but hard to read at a glance – avoid this style
result = [x * y for x in range(5) for y in range(5) if x != y if (x + y) % 2 == 0]
```

Rule of thumb: if you have to read a comprehension twice to understand it, write it as a plain loop instead. Readable code beats clever code.

Before You Move On — Self-Check

Use this list to confirm you're ready for Week 2 (OOP, Functions, Errors & Git). If any of these feel shaky, revisit that section before moving forward.

- ☐ I can install Python and set up a virtual environment, including activating and deactivating it
- ☐ I can point my editor at the correct interpreter
- ☐ I can use arithmetic operators and cast between int, float, str, and bool deliberately
- ☐ I can index, slice, and use common methods on strings, including escape sequences and f-strings, and I know strings are immutable
- ☐ I can format numbers with f-strings and `round()`, and I know why floats shouldn't be compared with `==`
- ☐ I can use `input()` and understand what `None` represents
- ☐ I can read a traceback and find the failing line, and recognize common exception types by name
- ☐ I understand the difference between `is` and `==`
- ☐ I understand operator precedence and short-circuit evaluation
- ☐ I can import a module three ways: whole module, specific names, and under an alias
- ☐ I understand which values are falsy in Python and why it matters
- ☐ I can write conditionals (including ternary expressions) and loops, including `enumerate()`, `zip()`, and `range()` with a step
- ☐ I can choose between a list, tuple, dict, or set for a given task
- ☐ I can use `.get()`, `.keys()`, `.values()`, `.items()`, and `del` on a dictionary
- ☐ I can use set operations (union, intersection, difference)
- ☐ I can use negative and step slicing, and work with a list of lists
- ☐ I can sort a list of dicts using `key=`
- ☐ I can read and modify a list of dicts and a dict of lists
- ☐ I can define a function with default arguments, keyword arguments, `*args` / `**kwargs`, and return multiple values
- ☐ I can write a docstring, use `pass` as a stub, and explain local vs. global scope
- ☐ I can rewrite a simple loop as a list or dict comprehension